

Autonomous Room Mapping with Crazyflie 2.1

Technical Project Report

Steven Van Norman

University of Central Florida

September 20, 2026



A technical report detailing the development of an unmanned aerial vehicle using the Crazyflie 2.1 nano-quadcopter, integrating MATLAB, Simulink, and Python

Table of Contents

1.0	ABSTRACT / SUMMARY	1
2.0	SYSTEM ARCHITECTURE	2
2.1	Hardware & Equipment	2
2.2	Communication Protocol & Architecture	2
3.0	SIMULINK MODELING	3
3.1	Quadcopter Dynamics Model	3
3.2	Solver Settings & Constraints	4
3.3	Altitude PID Tuning	4
4.0	MATLAB IMPLEMENTATION	5
4.1	Architecture & File Distribution	5
4.2	Flight Control Logic (drone_brain.m)	5
4.3	Command Hub & Live Mapping (command_hub.m)	6
5.0	TESTING	7
5.1	Ground Effect Mitigation	7
5.2	Telemetry Data Rate Optimization	7
5.3	Sensory Data Sanitization and Navigation Stabilization	7
6.0	RESULTS AND DISCUSSION	8
6.1	Flight Stability Analysis	8
6.2	Live 2D Mapping Performance	8-9
7.0	CONCLUSION	9
7.1	Project Summary	10
7.2	Testing Validation	10
7.3	Future Work: Hardware & Architecture Upgrades	10
7.4	Future Work: Software & Algorithm Expansion	11
8.0	REFERENCES	12

1 Abstract / Summary

The primary objective of this project was to design and implement an autonomous 2D room-mapping and telemetry architecture for the Crazyflie 2.1 nano-quadcopter within a MATLAB environment. The methodology utilized a Simulink model to refine control algorithms before deploying verified logic into two core modules: `drone_brain.m` for navigation and `command_hub.m` for wireless visualization via a Crazyradio link.

Flight testing successfully demonstrated robust autonomous operation across automated liftoff, stable hovering, obstacle-avoidant roaming, and landing sequences. The quadcopter effectively navigated indoor environments while maintaining a boundary margin and computing perpendicular escape vectors when paths were blocked. Hardware testing validated solid baseline stability, successfully managing environmental factors like ground effect through optimized landing thresholds. Furthermore, the custom telemetry dashboard successfully rendered real-time sensor streams into a live 2D room map without operational latency. Ultimately, this modular architecture establishes a reliable testbed for future advancements, such as 3D point-cloud mapping, onboard C-code execution, and advanced spatial navigation algorithms

2 System Architecture

2.1 Hardware & Equipment

The system utilizes the Crazyflie 2.1 STEM Ranging Bundle, which includes the following:

- **Crazyflie 2.1:** The primary micro quadcopter.
- **Crazyradio:** A transmitter used to send operational signals to the quadcopter.
- **Multi-ranger Deck:** A sensor array featuring 5-directional lasers (forward, backward, left, right, and upward) for obstacle detection.
- **Flow Deck:** A downward-facing sensor that measures distance from the ground using a cone-like laser.

The total weight of the drone with these additions is 0.0329 kilograms.

2.2 Communication Protocol & Architecture

The physical hardware interfaces with the software environment through a continuous telemetry link managed by the crazyradio. During flight operations, the Multi-ranger and Flow decks constantly gather spatial and altitude data. This information is transmitted wirelessly from the crazyflie to MATLAB via the crazyradio USB dongle. This continuous data pipeline is what allows the raw sensor inputs to be processed in real-time by the flight logic and visualized dynamically within my MATLAB environment.

3 Simulink Modeling

3.1 Quadcopter Dynamics Model

The “[Crazyflie Quadcopter Simulation Using Simmechanics](#)” was accessed through the MATLAB add-ons. The original file is provided above, and the modified version is presented below. Adding the custom MATLAB code block and completing the rewiring was straightforward. A constant block of “4” is used due to the laser's maximum distance of 4 meters.

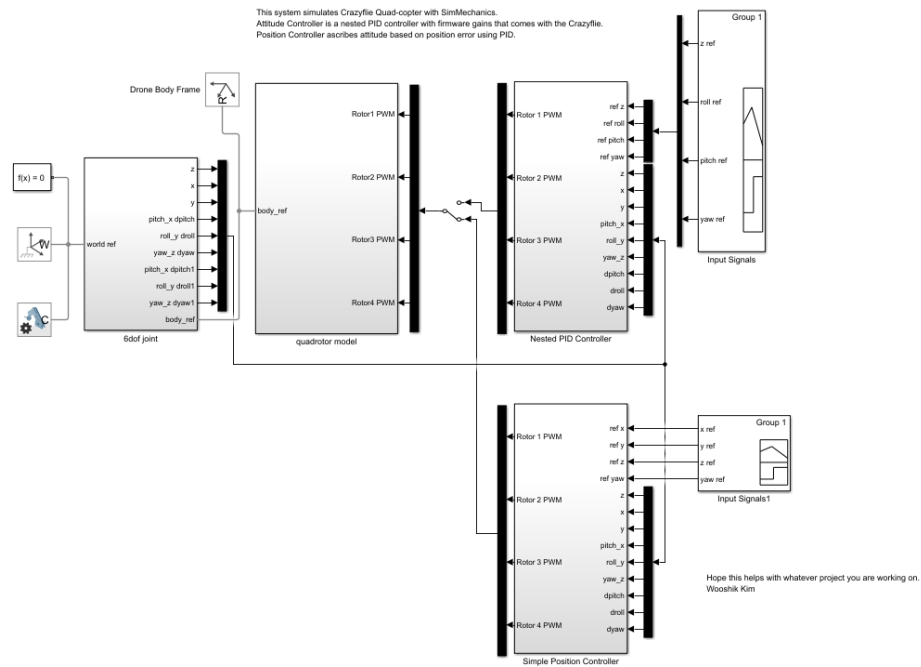


Fig 1: Crazyflie Quadcopter Simulation Using Simmechanics original file

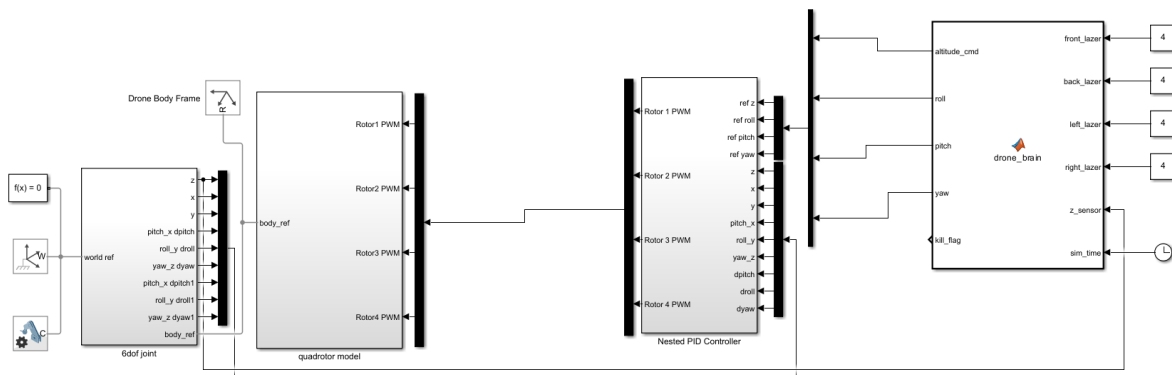


Fig 2: Modified Crazyflie Quadcopter Simulation Using Simmechanics file

3.2 Solver Settings & Constraints

The primary issue encountered while working in this Simulink environment involved attempting to switch the solver from a variable-step to a fixed-step configuration. This change was pursued because the quadcopter operates at 500 Hz, whereas a variable-step solver adjusts its step time as needed. Ensuring the Simulink model frequency matches the physical quadcopter hardware is crucial for avoiding zero-crossing.

However, switching to a fixed-step solver was not possible due to the Simscape blocks, which model physical components rather than pure mathematics. Considerable time was spent adjusting the Simscape block solvers, but this yielded no success. An attempt was also made to model a custom quadcopter capable of running on a fixed-step solver, but the associated PID controller proved overly complex. The final resolution was to allow Simulink to remain as a variable-step solver. This limitation is not expected to negatively impact the intended applications for the quadcopter.

3.3 Altitude PID Tuning

When the target z-value was set to a specific altitude, the quadcopter would linearly approach that target, increase thrust 0.14 meters early, reach an apex at the exact desired altitude, and then drop down to hover exactly 0.14 meters below it. This steady-state error was traced back to the altitude PID controller within the Simulink simulation. The system struggled to maintain the constant thrust required to counteract gravity, resulting in a permanent offset.

To resolve this, rather than relying on trial-and-error tuning, a characteristic equation based on the quadcopter's vertical flight dynamics was used to calculate the optimal theoretical values for the Proportional, Integral, and Derivative gains. Applying these calculated values back into the simulation provided the necessary integral action to completely eliminate the steady-state error, allowing the quadcopter to smoothly hold the exact target altitude without any undershoot or overshoot.

4 MATLAB Implementation

4.1 Architecture & File Distribution

This project utilizes two MATLAB code files: `drone_brain.m` and `command_hub.m`. All logic dictating how the quadcopter lifts off, hovers, roams, lands, and avoids obstacles is contained within `drone_brain.m`. The bridge to the Python library used by the Crazyflie 2.1 is located in `command_hub.m`, which also generates the 2D mapping figure.

4.2 Flight Control Logic ([`drone\_brain.m`](#))

The quadcopter features four flight modes: liftoff, hover, roaming, and landing. The functions for these modes are located at the end of the code file. Persistent variables are utilized for some of the logic to ensure the quadcopter retains values between function calls. This is crucial for the roaming function, preventing the quadcopter from returning to its previous location after encountering an obstacle. On startup, the quadcopter ascends to the target z-value and then switches to hover mode. In hover mode, it scans for the closest wall and, after a set duration, roams toward it, at which point the roaming logic takes over.

During the roaming phase, the system utilizes sensor data from four directional lasers (front, back, left, and right) to safely navigate the environment. Initially, the quadcopter identifies the closest wall and locks onto that direction using a persistent variable. During flight, a roaming speed of 0.5 meters per second is maintained, alongside a safety margin of 0.5 meters from any obstacle. If the quadcopter detects that its current path is blocked within this margin, it automatically calculates a new, perpendicular flight path. For instance, if forward or backward movement is obstructed, it evaluates the lateral laser distances and pivots right or left depending on which direction offers more open space. If all perpendicular paths are also blocked, the system commands the quadcopter to reverse its course. The selected direction is then directly translated into specific roll, pitch, yaw, and altitude commands to maintain continuous, collision-free movement.

4.3 Command Hub & Live Mapping ([command_hub.m](#))

The `command_hub.m` script serves as the central control, communication, and visualization hub, bridging MATLAB with the quadcopter firmware over the Crazyradio protocol. To optimize the low-level integration process, artificial intelligence tools were leveraged to analyze the firmware structure and align the corresponding telemetry variables. Utilizing AI for this task streamlined the identification of exact variable names, data types, and byte constraints, converting a tedious data-mapping process into an efficient development workflow.

Architecturally, the script dynamically generates a Python script (`cf_telemetry.py`) to interface directly with the native Crazyflie library (`cflib`). Because the Crazyradio protocol restricts individual logging packet payloads to approximately 26 bytes, the telemetry configuration was strategically divided into two distinct logging blocks—one for laser sensor ranges and altitude, and another for horizontal position and velocity vectors. Both streams run on a 20 ms sampling interval to maintain real-time telemetry feed quality.

The incoming sensor stream is continuously passed to `drone_brain.m` for setpoint calculations while simultaneously updating a live graphical interface. This dashboard displays the quadcopter's real-time position, flight trajectory, active mode status, velocity, and spatial wall returns using optimized rendering (`drawnow limitrate`) to avoid introducing latency into the main flight loop. Finally, an `onCleanup` routine is included as a safety precaution to ensure the quadcopter executes an immediate shutdown command and closes the radio link if the program is interrupted or encounters an error.

5 Testing

5.1 Ground Effect Mitigation

The most significant discrepancy between Simulink modeling and physical quadcopter flight in a confined testing area was the influence of ground effect. Given the low mass of the quadcopter (32.9 grams), ground effect noticeably impacted flight stability near surfaces. The workaround was to program the quadcopter to disarm the motors once a specific threshold height was reached during Case 4 (the landing phase). To determine this threshold, a temporary `disp()` statement was added in MATLAB to monitor altitude fluctuations during landing attempts. The system was observed until instability occurred, and the maximum recorded fluctuation height was determined to be 0.03 meters.

5.2 Telemetry Data Rate Optimization

A basic display hub was initially implemented within `drone_brain.m` to troubleshoot an issue where the quadcopter bypassed hover mode, transitioning directly from liftoff to landing. While this display functioned properly in Simulink, hardware testing caused telemetry data to output every few milliseconds, cluttering the MATLAB command window. Throttling the output rate to 0.5-second intervals resolved the command window clutter; however, separating the control logic from the visual figures provided a cleaner, more modular architecture. Consequently, telemetry figures and dashboard displays were migrated to `command_hub.m`, leaving `drone_brain.m` focused entirely on core control logic.

5.3 Sensor Data Sanitization and Navigation Stabilization

During flight testing, an intermittent navigation anomaly was observed where the quadcopter would fail to move laterally after encountering a wall and instead retreat to the previous direction of travel. This occurred due to an underlying hardware bug where readings exceeding four meters returned a value of zero instead of a standard NaN, these zero-readings were falsely interpreted as immediate, zero-distance obstacles. This corrupted the directional evaluation, causing the decision-making logic to misidentify open paths and steer the drone back in the direction it came from rather than executing a smooth lateral transition.

To resolve this vulnerability and ensure robust state transitions, the algorithm overrides the value to a maximum default distance, correctly classifying the signal as open space rather than a physical obstruction. This critical patch eliminated false obstacle triggers, stabilized the directional decision-making process, and allowed the drone to smoothly navigate tight corridors without unwanted reversals or erratic behavior.

6 RESULTS AND DISCUSSION

6.1 Flight Stability Analysis

Flight testing demonstrated strong operational stability across all phases, while also highlighting notable differences between the simulated environment and physical hardware performance. In the preliminary Simulink simulation, the quadcopter model exhibited a 0.14-meter steady-state offset during vertical altitude holds, which required mathematical PID tuning to resolve in software. However, physical flight tests revealed that the actual quadcopter hardware did not suffer from this simulation artifact; the physical aircraft linearly approached target altitudes and maintained stable hovers without experiencing the altitude jump, overshoot, or undershoot predicted by the un-tuned model.

State transitions across all four flight modes—liftoff, hover, roaming, and landing—executed reliably in hardware. During autonomous roaming, pitch and roll velocity setpoints of 0.5 meters per second provided smooth directional adjustments without destabilizing the frame or causing unexpected altitude loss. Ground effect served as the primary physical disturbance affecting physical flight due to the quadcopter's low mass (32.9 grams). Implementing a motor disarm threshold at an altitude of 0.03 meters in the landing logic successfully bypassed low-altitude aerodynamic turbulence, ensuring safe touchdowns. Ultimately, comparing the software simulation to physical flight highlighted a classic sim-to-real discrepancy, proving that while Simulink required PID gain adjustments to model physical dynamics accurately, the physical onboard flight controller maintained solid baseline flight stability throughout testing.

6.2 Live 2D Mapping Performance

To transition away from basic interactive command-window testing and provide comprehensive spatial awareness, a custom visualization and telemetry dashboard was developed to monitor the quadcopter's flight performance in real time. This interface plots the drone's continuous flight path as a blue line alongside aggregated laser rangefinder endpoints that merge to outline the true structural perimeter of the room in red. Positioned alongside the spatial map, the diagnostic side panel displays crucial real-time testing parameters—such as the active flight mode, Cartesian coordinates, velocity vectors, individual laser distances, and elapsed flight time—which served as an essential tool for debugging and validating the autonomous navigation algorithms during active flight.

However, closer analysis of the environmental point cloud reveals minor geometric discrepancies, particularly manifested as slight deviations from straight boundary lines and incomplete data capture within the room's corners. These imperfections are primarily attributed to the physical mass and dynamic momentum of the quadcopter during flight; as the vehicle executes maneuvers, its weight induces minor inertial banking and attitude shifts that tilt the rigidly mounted laser rangefinders, broadening the point cloud distribution.

A viable engineering solution to mitigate these mapping errors and improve data fidelity is to implement active heading control, allowing the drone to rotate and continuously face its direction of travel rather than maintaining a static orientation from liftoff. By dynamically adjusting the yaw axis so that the vehicle's primary forward vector aligns directly with its movement vector, the system would minimize off-axis projection distortion caused by lateral roll commands and enable sharper, more accurate resolution of internal room corners.

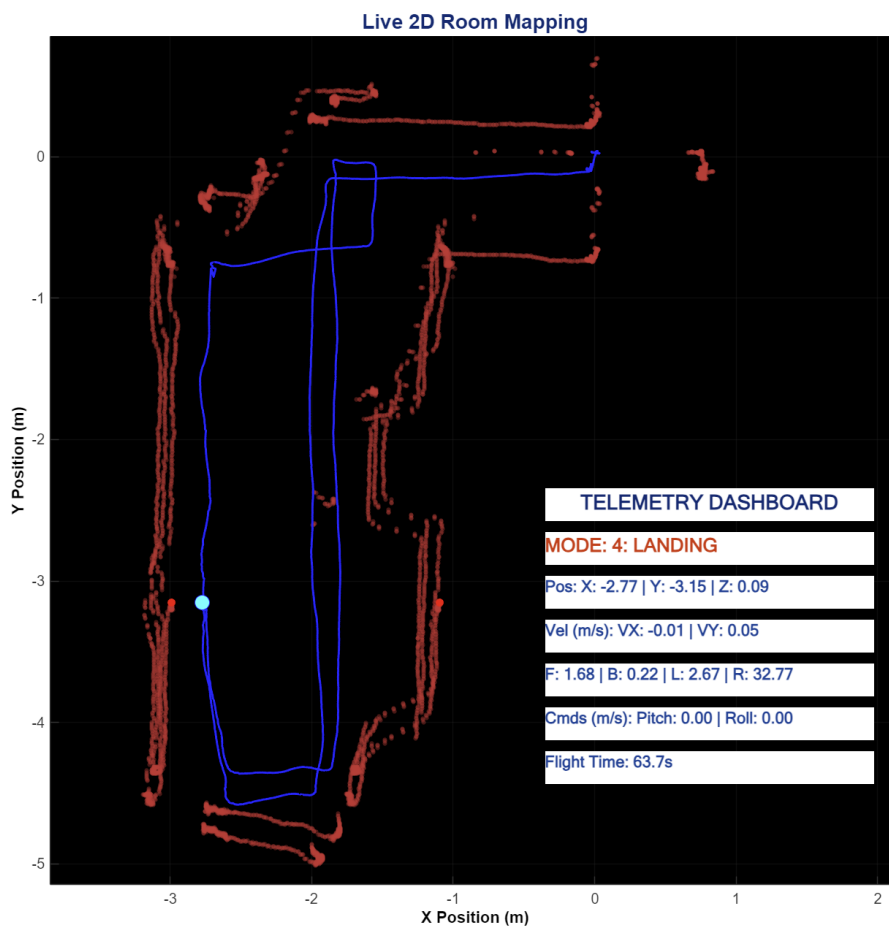


Fig 3: Live 2D room mapping result and telemetry dashboard display

7 CONCLUSION

7.1 Project Summary

The project successfully demonstrated the autonomous operation and real-time telemetry tracking of a Crazyflie 2.1 quadcopter using a custom MATLAB control architecture. By dividing the system into two distinct operational modules—`drone_brain.m` for core flight logic and `command_hub.m` for data visualization—the quadcopter reliably executed liftoff, stable hovering, obstacle-avoidant roaming, and automated landing sequences.

Leveraging artificial intelligence to map firmware variables streamlined the integration of the Crazyradio protocol, facilitating a continuous telemetry stream. This allowed raw distance and position data to be plotted into a live 2D room map without introducing latency into the primary flight loop.

7.2 Testing Validation

Testing and analysis validated the simulation against real-life dynamics. The quadcopter successfully navigated its environment by maintaining a 0.5-meter boundary margin, accurately calculating perpendicular escape vectors when forward or lateral progression was obstructed. Additionally, the development process underscored the limitations of software simulation compared to physical hardware; while the Simulink environment necessitated theoretical PID tuning to correct a simulated 0.14-meter altitude offset, the physical flight controller maintained stable, linear altitude tracking natively. Finally, the identification of ground-effect turbulence affecting the quadcopter led to a successful mitigation strategy, where an automatic motor disarm threshold at 0.03 meters ensured consistent and safe touchdowns.

7.3 Future Work: Hardware & Architecture Upgrades

Future iterations of this platform could focus on expanding the telemetry visualization and optimizing the control loop execution. The live 2D spatial mapping could be upgraded to a full 3D point-cloud environment by fusing pitch, roll, and altitude telemetry with the existing laser range data. Furthermore, while the MATLAB-driven variable-step solver provided sufficient performance for local area navigation, transitioning the control architecture to a fixed-step solver via C-code generation would allow the logic to run natively on the quadcopter's onboard micro-controller. This embedded approach would eliminate reliance on the continuous radio link, enabling higher-frequency control loops and more complex autonomous path planning in larger, unstructured environments.

7.4 Future Work: Software & Algorithm Expansion

Furthermore, additions to `drone_brain.m` can be readily implemented due to the highly modular architecture of the flight control logic. Future developers and researchers can leverage this foundational code by altering existing parameters or introducing entirely new flight states beyond the standard liftoff, hover, roam, and landing sequence. For instance, the autonomous roaming function can be significantly expanded; rather than relying solely on reactive, perpendicular turns upon encountering an obstacle, the logic could be upgraded to execute structured spatial algorithms. This might include programming grid-based "lawnmower" search patterns to systematically sweep and map a room, or implementing wall-following behaviors for perimeter tracking and maze navigation. Additionally, by adjusting baseline variables—such as tightening the 0.5-meter safety margin or altering the 0.5 meters per second velocity limit—the code can be easily tailored to safely navigate highly confined environments or conduct rapid area exploration. Ultimately, the separation of the decision-making logic from the hardware communication establishes `drone_brain.m` as a versatile, open-ended testbed for more advanced autonomous navigation research.

References

- [1] W. Kim, "Crazyflie Quadcopter Simulation Using Simmechanics," *MATLAB Central File Exchange*, 2017. [Online]. Available: <https://www.mathworks.com/matlabcentral/fileexchange/65469-crazyflie-quadcopter-simulation-using-simmechanics>. [Accessed: Sep. 2026].
- [2] S. Van Norman, "command_hub.m," *Autonomous-Room-Mapping-with-Crazyflie-2.1*, GitHub repository, 2026. [Online]. Available: https://github.com/StevenVanNorman/Autonomous-Room-Mapping-with-Crazyflie-2.1/blob/main/command_hub.m. [Accessed: Sep. 2026].
- [3] S. Van Norman, "drone_brain.m," *Autonomous-Room-Mapping-with-Crazyflie-2.1*, GitHub repository, 2026. [Online]. Available: https://github.com/StevenVanNorman/Autonomous-Room-Mapping-with-Crazyflie-2.1/blob/main/drone_brain.m. [Accessed: Sep. 2026].